

Theoretical Questions

July 19, 2024

1. (a) $\{f : [T2 \rightarrow T3], g : [T1 \rightarrow T2], a : \text{Number}\} \vdash (f(ga)) : T3$

Answer: True

Given f is a function from $T2$ to $T3$ and g is a function from $T1$ to $T2$, and a is of type Number (which we assume to be of type $T1$), applying g to a yields a result of type $T2$. Subsequently, applying f to this result yields a type $T3$.

- (b) $\{f : [T1 \rightarrow [T2 \rightarrow \text{Boolean}]], x : T1, y : T2\} \vdash (fxy) : \text{Boolean}$

Answer: False

The expression fxy is incorrectly parenthesized. f is a function that takes an argument of type $T1$ and returns another function from $T2$ to Boolean . The correct application should be $(fx)y$, which applies f to x first, resulting in a function from $T2$ to Boolean , and then applies this function to y , resulting in a Boolean . Therefore, fxy does not directly yield a Boolean .

- (c) $\{f : [T1 \times T2 \rightarrow T3], y : T2\} \vdash (\lambda(x)(fxy)) : [T1 \rightarrow T3]$

Answer: True

The lambda expression $\lambda(x)(fxy)$ defines a function that takes an argument x of type $T1$ and applies the function f to the pair (x, y) . Since f takes a pair $(T1, T2)$ and returns $T3$, the lambda expression is indeed of type $[T1 \rightarrow T3]$.

- (d) $\{f : [T2 \rightarrow T1], x : T1, y : T3\} \vdash (fx) : T1$

Answer: False

The function f expects an argument of type $T2$, but x is of type $T1$. Therefore, the expression fx is not well-typed because x does not match the expected input type for f .

2. (a) i. $(\text{inter number boolean}) \rightarrow \text{never}$
ii. $(\text{inter any string}) \rightarrow \text{string}$
iii. $(\text{union any never}) \rightarrow \text{any}$
iv. $(\text{diff (union number string) string}) \rightarrow \text{number}$
v. $(\text{diff string (union number string)}) \rightarrow \text{never}$
vi. $(\text{inter (union boolean number) (union boolean (diff string never))}) \rightarrow \text{boolean}$

```

(b) ;; L52 return type is? boolean
    ;; L51 return type boolean
    (define (isBoolean : (any -> is? boolean))
      (lambda ((x : any)) : is? boolean
        (boolean? x)))

    ;; Function to complete
    (define (good_in_L52 : ((union number boolean) -> [a]))
      (lambda ((z : (union number boolean))) : [b]
        [c]
      ))

    ;; Completed
    (define (good_in_L52 : ((union number boolean) -> boolean))
      (lambda ((z : (union number boolean))) : boolean
        (isBoolean z)
      ))

```

- (c) **The return type for f should be (union string one).** The function f checks if x is a number and then checks if it is positive or negative, returning the strings "positive" or "negative" respectively. If x is a boolean, it returns the boolean value itself. If x is neither, it returns 1. Thus, the possible return types are either a string or the number 1, which can be represented as (union string one).